

LLM Generative AI and Students' Exam Code Evaluation: Qualitative and Quantitative Analysis

E. Smolić*, M. Pavelić†, B. Boras*, I. Mekterović*, T. Jaguš†*

* University of Zagreb, Faculty of Electrical Engineering and Computing, Department of Applied Computing

† Šibenik University of Applied Sciences

{ema.smolic, bartol.boras, igor.mekterovic, tomlav.jagust}@fer.hr; marko.pavelic@vus.hr

Abstract—Since the introduction of generative artificial intelligence (GAI) technology in the context of large language models (LLMs), it has been widely used for information extraction and/or extrapolation from different sources. In computer science education, a potential application of such technology is for automatic code review, i. e. shifting the burden of debugging non-compilable code, detecting overlooked optimization concerns such as poor memory management in code that otherwise passes automated tests, and other advanced tasks from a human grader to LLMs. However, LLMs are currently not capable of evaluating code or mathematical expressions with 100% reliability, i. e. beyond token pattern recognition and subsequent probabilistic answer generation. With that in mind, in this paper, we explore the risk of incorrect LLM code evaluation, both descriptive and numerical, as well as begin research on its mitigation and propose further work directions.

Keywords—LLM, code evaluation, supported grading

I. INTRODUCTION

Following its recent rapid development, generative artificial intelligence (GAI) based on large language models (LLMs) has been widely employed covering a broad range of use-cases involving text generation: from coding to creative or scientific writing. The vast knowledge base contained in LLMs combined with non-deterministic output computation has shown to be especially useful when seeking to consider a problem from a different perspective or gain additional insight. Meanwhile, although extensively used for code generation, research on the technology's ability to analyze and evaluate code appears to be remarkably scarce. Furthermore, in the rare studies on the evaluation ability, it was shown to be underdeveloped, especially when compared to performance on natural language [1].

A common practice in LLMs designed to work with code involves the use of built-in or external compilers, able to warn about issues such as non-compilability or functional errors. However, the main emphasis of this research is placed on qualitative assessment, for instance, detection of sub-optimal memory management, unnecessarily high complexity, or even style-related issues such as lack of clarity in variable names, which is not detectable through solely compilation and running.

The aim of this research is to 1) assess the quality of code review performed by state-of-the-art LLM GAI compared to human graders and 2) explore and comment on the benefits and drawbacks of the few-shot prompting method on task performance. The topics are researched

through two respective experiments on SOTA commercial general LLMs: the first one testing code review ability using zero-shot prompting and the second one seeking to address problems arising in the former through few-shot prompting.

Therefore, we employ freely available web interfaces for interaction with specific commercial general LLMs and present experimental data via prompting. Received output is collected and evaluated by human experts according to specific experiment requirements, as well as analyzed/commented on.

The results suggest that 1a) commercial general LLMs can be useful in providing supplementary insight on code, but with required human supervision, as is supported by related research; 1b) automatized score assignment by those LLMs based on code quality is at best comparable to random assignment with multiple possible explanations; and 2) few-shot prompting for the task of code grading better adjusted to preferential criteria can work in a very limited scope.

II. RELATED WORK

The main motivation behind this research is the optimization of the programming assignment grading process. Specifically, we explore the prospect of a tool serving as a grading assistant offering additional insight. Following the latest advances in the field of LLM technology with application in GAI, results indicate they are suitable for a number of tasks related to code analysis, bug/vulnerability detection, and quality assessment. In this section, we present background work on these three points, respectively, as well as discussion on a similar recent study.

A human evaluator is likely to benefit from automatic code break-down. For example, one may encounter poorly organized projects they struggle to understand thoroughly in a reasonable time frame. As of now, available LLM technology holds an ambivalent reputation regarding its performance on code explanation and analysis. On the one hand, a study presented in [2] found that LLMs perform significantly better on the task of code explanation when compared to engineering students, taking into account both understandability and accuracy. Consequentially, it seems reasonable to speculate that using an LLM assistant could facilitate the review process to a degree via explanation provision. Moreover, LLMs have also been researched in

the context of automatic concept identification specially targeted towards programming assignments [3], where they exhibited potential. On the other hand, results described in [1] indicate LLMs' inferiority in answering multiple-choice questions involving program code as opposed to purely natural language problems. Also, the paper emphasizes the problem of insufficient research on LLMs in code analysis in the educational context, and the authors even propose future work involving the generation of explanations along with answers - the approach implemented in this study. Nevertheless, we employ explanations to facilitate answering on open-ended questions.

Code language models (CLM), which are language models trained on large corpora of program code and most extensively used for code generation, have been researched for the task of automatic program repair (APR) [4]. They have been found to surpass other methods e. g. those based on deep learning [5]. Moreover, fine-tuning the model yielded a considerable improvement in APR tasks. Additionally, following the rise of commercial general LLMs, studies have been conducted exploring the suitability of out-of-the-box general LLMs for debugging. Results range from promising with required additional validation [6], [7] to even surpassing widely used alternative approaches [8]. Further, LLMs fine-tuned on code repair data can outperform LLM baselines [9], posing grounds for few-shot learning and proposed future work. Also, LLM fine-tuning advances have been proven effective for the debugging task [10]. Finally, advances have been made in the field of LLMs for holistic code review, i. e. not exclusively vulnerability or bug detection, such as [11], which is a *GitHub* repository claimed to still be in alpha stage and not to be used for critical assignments, but to the best of our knowledge have not resulted in peer-reviewed scientific papers. More closely related to our work, a study has been conducted exploring the use of LLMs for bug identification in introductory programming assignments [12]. Similar to this study, the researchers presented an LLM (GPT-3.5 turbo) with a few dozen instances of student-written code asking it to detect errors, and subsequently compared the collected output to manual assessments generated by human teaching assistants. While finding the LLM performance less than optimal in terms of precision and recall, the authors recognize the advisory potential of such tools for skilled TAs able to filter out potential hallucinations and errors. However, the study required test examples to already be syntactically correct, which we do not and therefore include syntax error detection into our research scope. Moreover, our study goes a step further and conducts experiments involving few-shot learning and, for future work, fine-tuning. Alternatively, another study [13] showed that, when acting as a diagnostic tool on strictly erroneous student code required to adhere to task specification, LLMs mostly offer at least one valid bug report, but also generate many false positives. Meanwhile, our research includes potentially entirely correct code, where there is no chance of LLMs guessing a bug diagnosis merely through a numbers game. Finally, the paper specu-

lates on the potential of customized LLMs for such tasks, which we build upon by adjustment experiments. Although LLMs are currently being extensively studied for code diagnostics and debugging, they still showcase significant limitations. Nevertheless, our specific use case may just mitigate them and therefore increase their reliability and usefulness.

Furthermore, research on the suitability of LLMs for evaluation of specifically student-written code has undergone a recent increase. Results obtained in [14], a study involving automated assessment of student programming assignments through GPT-3, suggest that the model is currently too unreliable to act as a direct feedback provider for students. Concretely, the problems lie in its high variability in grade accuracy, as well as feedback inconsistency. While its experimental setup is almost identical to the one used in this paper, the research questions are different: instead of focusing solely on accuracy and consistency essential for unsupervised automatized grading, we also put emphasis on novel insight. That is, we find it acceptable that tolerating some inaccuracy, which will ultimately pass through a teaching assistant acting as a human-in-the-loop, may lead to a broader assessment scope highlighting issues the human grader might have overlooked, and evaluate model output accordingly. Research showcased in [15], which is also concerned with evaluating LLM-generated feedback to student programming assignment solutions, draws some interesting conclusions on assessment quality over mistake types contained in the presented code. The authors of the aforementioned paper noted a lack of concern for code intention as their limitation. For that reason, this research included the ability to grade code against task specification, imposing additional assessment criteria. Moreover, we also seek overall score suggestions, which the paper is not focused on. In other words, our research is more aimed towards correctness and quality evaluation based on both task requirements and general best practices, as well as providing a calculated grade. Another study, presented in [16] considers problem-specific requirement adherence alongside isolated quality concerns, as well as including a binary correct/incorrect predicate with a descriptive assessment. Recognizing that more informative grading (e.g. percentage score instead of pass/fail) is an additional task providing a large benefit in our use case, we diverge from the aforementioned binary approach and thus extend the research significantly. Lastly, experiments undertaken in [17], while also assessing both code correctness and quality, are limited to rubric grading and thus differ from our open-ended approach.

Finally, during the production of this paper, another study [18] was submitted for review, which bears striking resemblance to ours in terms of the experimental procedure, i.e. comparison of zero-shot and few-shot prompting. However, the study focuses on generating code fixes. We, however, require commentary as the main focus and even disregard accidentally obtained code fixes, as they are not strictly included in the use case. Therefore, we draw from

newly presented knowledge on shared procedure steps and adjust the methodology accordingly, applying it to a complimentary use case.

III. METHODOLOGY

The models included in testing are commercial general LLMs. Namely, the choice fell on GPT-3.5 and Gemini. Initially, PaLM 2 was in consideration, but preliminary research showed it unsuitable for our experiment due to its refusal to generate numerical scores.

The data available prior to experiment execution consisted of 50 instances of simple C code, submitted by introductory programming course students as assignment solutions, along with TA-assigned points (converted to percentage scores for easier comparison) and commentary for each code instance. The sample contained no personal information regarding students.

A. Zero-Shot Prompting

The goal of the first experiment was twofold: 1) to collect LLM-generated commentary and grading of individual programming assignments, and 2) to collect human experts' numerical evaluation of the LLM-based commentary and grading.

The first goal was executed through conversation with the LLMs, employing their respective chat-based web applications. The conversation flow is illustrated in Table I. Grading and comments were obtained by first providing a natural language task explanation as the initial prompt, an approach known as zero-shot prompting [19]. Then, the LLM was presented with code instances excluding any additional instruction or commentary, to which it replied with an assessment. Exceptionally, sometimes extra scaffolding was necessary (e. g. to include percentage score or return the LLM to the task). Additionally, persona prompting was omitted, which is supported by [18], where the practice was found harmful to LLM performance on related code repair tasks. A goal in this step was to obtain an independent and highly non-deterministic (but still correct) assessment, for which persona prompting could be counterproductive.

After instruction, code instances were presented one by one, with a new presentation immediately following the LLM response on its predecessor, spare the exceptional cases such as the scaffolding described above.

After data collection, three researchers individually graded a) comment accuracy (with TA comments for easier orientation) and b) the correspondence of the assigned score to the generated comment. It is essential to note that the human experts did not directly evaluate the relationship between LLM-generated percentage score and that assigned by a TA, which is addressed by a purely statistical analysis. The grading was based on previously established guidelines and consisted of integer scales 1 - 5 and 1 - 3 respectively. A higher grade indicates a more favorable outcome. Finally, the compiled data underwent a statistical analysis, which is discussed in the Results and Discussion section.

TABLE I
AN EXAMPLE OF CONVERSATION

Prompt (<i>Task instruction</i>): You are a grader grading student-written code assignments. I will present C programs one by one, and you will respond to each with a quality assessment expressed in percentage score, with 100% being flawless and fully optimal code, as well as commentary on your grading. The code should find and print all prime numbers in the range from 100 to 999 that have a specified input digit as one of their digits. Then, it calculates and prints the sum of these prime numbers.
LLM : I can certainly explain code and review its functionality and optimization concerns. Please provide me with the code you would like me to review.
Prompt : <pre> #include<stdio.h> #include<stdlib.h> int prim [1000]; ... return 0; } </pre>
LLM : The provided code seems to be well-structured and efficiently implemented. Here's a breakdown of what the code does: 1. It initializes an array 'prim' of size 1000 to store prime numbers. ... Overall, the code efficiently generates prime numbers up to 999 and correctly calculates the sum of prime numbers based on the user's input digit. It appears to be well-written and accomplishes its task effectively. Based on the provided code's clarity, efficiency, and correctness, I would assign it a grade of 95%. While the code is well-structured, efficiently implemented, and achieves its intended functionality, there is some room for improvement in terms of readability and commenting. ... Overall, it's a well-written program with only minor areas for enhancement.

B. Few-shot prompting

In an attempt to balance out LLMs autonomy on answer design and score computation with own preferences, we experimented with few-shot learning [19]. This technique based on the presentation of output examples during prompting, provides additional context without weight correction. According to the paper [19], the few-shot approach on GPT-3 can yield performance comparable to that of state-of-the-art fine-tuned models on various NLP tasks. However, we experimented on Gemini due to its significantly worse performance compared to GPT-3.5, which renders improvement more apparent and verifiable.

To test if this approach causes an LLM to generate output better matching our grading criteria, while still allowing it to include additional points, K = 11 examples out of the initial 50 were selected, which include code, percentage score, and commentary. The examples we had access to are in the Croatian language, but as the LLMs included in research are also able to generate Croatian text as well as translate from English to Croatian, the premise that the models should be able to also generate code review in Croatian was accepted. The predicate for example selection was that the absolute difference between the ground-truth score and the score assigned by Gemini equals at least 45%.

Subsequently, they were alternated between serving as an example and test instance in the form as illustrated in Figure 1. The experimental procedure was not rigidly

defined and was instead more heuristic-oriented, and also allowed for additional prompting for scaffolding.

```
"student_answer_code": "#include
<stdio.h>...int isPrime(int broj)
{...} ...",
"comment": "...
Komentari su sljedeći:
- Nisu otklonjene trivijalne
sintaksne pogreške. ...",
"score": 0

...

"student_answer_code": "#include
<stdio.h>
..int main(void) {\n    int n,
zbroj = 0, i, j, ... } ...",
"comment":
```

Fig. 1. Example of a few-shot learning prompt

IV. RESULTS AND DISCUSSION

A. Zero-Shot Prompting

The following data was obtained 1) comments and grades (expressed as percentage scores) on 50 programming assignments generated independently by two LLMs; and 2) human expert numerical evaluation of two LLMs respective performance on code commentary and commentary-score correspondence.

The human expert evaluations reached an inter-rater reliability (IRR) consensus evaluated by Krippendorff's alpha, with the threshold set to $\alpha = .8$. The final scores were as follows:

- **GPT-3.5**
commentary quality: $\alpha = .878$
commentary-score correspondence: $\alpha = .859$
- **Gemini**
commentary quality: $\alpha = .928$
commentary-score correspondence: $\alpha = .87$.

Table II presents an overview of the human expert LLM-output evaluation.

TABLE II
STATISTICAL OVERVIEW OF HUMAN EXPERT LLM-OUTPUT
EVALUATION

	GPT3.5 comm.	GPT3.5 score from comm.	Gemini comm.	Gemini score from comm.
Min	1	1	1	1
1st Qu.	3.25	2.00	2.00	2.00
Median	4	3	4	3
Mean	4.067	2.62	3.5	2.54
3rd Qu.	5.0	3.0	4.0	3.0
Max	5	3	5	3

Based on available data, several conclusions can be drawn:

- 1) GPT-3.5 appears to perform better on commentary generation than Gemini both on average and according to low-quality output frequency;

- 2) both models seem to assign scores justified by issues addressed in the generated commentary.

An interpretation of these findings is that these commercial general LLMs can provide code commentary which, although not perfect by itself, addresses valid issues. In an educational context, that means graders can depend on such LLMs for initial orientation, faster code comprehension, and double-checking for possible missed observations, as it is a usual occurrence (at least half of the time) for a commentary to be of *very good* (4) or even *excellent* (5) quality. Nevertheless, zero-shot prompted unspecialized LLMs did not prove reliable enough, neither for automatic feedback generation nor utilization by non-specialist graders unable to detect hallucinations, which is reflected in the relatively high frequency (for Gemini at least 25%, for GPT-3.5 a little less) of commentary not even reaching the level of *good* (3). In conclusion, they often offer a good starting point, but require validation or additional scaffolding, as is supported by studies [6], [12]–[14].

Concerning score-commentary correspondence, the obvious conclusions drawn from this measure would be those regarding the ability of subject LLMs to either display consistency in two different tasks based on the same premise, infer a numerical measure from a text-based critique (or even vice versa), or recognize an input pattern mapping to both the commentary and the score with such confidence that the intra-output token dependence becomes nearly irrelevant. However, one less overt finding relates to the problem of LLMs, and, more broadly, AI, lacking immediate common-sense insight into pedagogical and interdisciplinary concepts. For illustration, one should consider a recurrent practice of LLMs proposing a score equaling the mathematical average of separate scores assigned for rubrics such as *Correctness*, *Efficiency*, *Clarity* and *Style* observed in the experiment. As a consequence, minor stylistic concerns such as variable nomenclature or a lack of clarifying comments, which result in significantly lower scores for respective rubrics, have the same detrimental impact on the overall grade as logical mistakes or unacceptably high time complexity. While worth noting and possibly justifying a slight reduction in score, it is neither pedagogical nor grounded in knowledge of the field to regard listed types of mishaps as equally destructive (except for cases where, for example, the assignment focuses on documentation skills). Mitigating such practice would require detailed and carefully constructed pre-prompting as well as constant additional scaffolding, which has an unfavorable effect on tool flexibility and its potential practical use. Instead, this observation was used as a motivation for the few-shot prompting approach, where the model was tested on structure and priority inference based on a small number of examples. Finally, while not immediately apparent from data showcased in Table II, from which one can conclude (based on the median of 3 and a high mean), that the described problem was not shown to be prominent in this setup, we have encountered the phenomenon frequently. The lack of nuance in grades, originally selected

to include only three degrees for the sake of flexibility (i. e. a difference between commentary deserving 60% and 70% is likely to be nearly imperceptible for a single expert, with the complexity rising with multiple ones) and represent respectively (*almost*) *no correspondence*; *somewhat acceptable correspondence, but not the expert's first instinct*; and *alright to full correspondence* has shown to be a hindrance to explicit illustration of the phenomenon. Therefore, more insight into the described behavior in the few-shot prompting experiment was provided, as well as proposed research in specialized fine-tuned models as future work.

Finally, the assigned score alignment between human graders and the two models was addressed. An overview of relevant data is provided in Table III. Note that, due to GPT-3.5's refusal to generate scores for three code instances, their data has been omitted in the following analysis. For full transparency, their scores are provided in the form of (human-assigned, Gemini-assigned): (0%, 40%), (0%, 60%), (100%, 50%). Also, these examples alone indicate Gemini's suboptimal performance on the task, which will be statistically supported later in the section.

TABLE III
STATISTICAL OVERVIEW OF SCORES ASSIGNED BY HUMAN GRADER AND LLMs

	human	GPT-3.5	Gemini
Min.	0.00	15.00	20.00
1st Qu.	25.00	68.50	65.00
Median	80.00	85.00	75.00
Mean	62.41	77.11	73.32
3rd Qu.	100.00	93.00	85.00
Max.	100.00	100.00	98.00

The data indicate that:

- 1) zero-shot prompted general LLMs appear significantly less inclined to assign very low scores than an (objective) human grader,
- 2) nevertheless, a higher concentration of grades in the mid-high range poses as a stark difference to the human grader's decision to place only about a quarter of submissions into the vast 25% - 80% range,
- 3) zero-shot prompted general LLMs seem prone to score reduction when encountering code marked as fully correct by a human grader.

The first and the last point, provided they are a consequence of a mostly semantically-driven relationship between code and its assigned score, which is both likely but also grounds for future research, could be a consequence of the problem of insufficient insight in, and suboptimal alignment with grading standards employed by the human grader, which was described earlier in this section. Moreover, the second point possibly indicates an error-minimization approach of gravitating towards 50%, but drawing definite conclusions would, again, require a more detailed study on the score generation mechanism.

However, checking Pearson's paired r-coefficient, which measures linear data correlation between paired samples,

on scores assigned to the same code instance by different graders, showed the scores to have a rather suboptimal relationship. The computed coefficients are displayed in Table IV. It should be noted that, while Pearson's r-coefficient does not determine the correlation of order higher than linear, the relationship between scores assigned to the same prompt must indeed be linear.

TABLE IV
ASSIGNED GRADES' PEARSON'S R-COEFFICIENTS

	GPT-3.5	Gemini
human grader	.6443089	.230244
GPT-3.5		.4299363

Only scores assigned by GPT-3.5 exhibit somewhat significant (but still unacceptable for this use-case) correlation to those assigned by a human grader, while that of Gemini is low enough for the scores to be assigned by chance - which is reflected in the scores for three omitted examples noted earlier. Additionally, GPT-3.5's and Gemini's scores correlate to a degree too low to be considered adequate for this application. Therefore, it can be concluded that score assignment by LLMs is ineffective and requires an adjusted prompting technique, model, or some other approach.

B. Few-shot prompting

A side-by-side comparison of human grading and pre- and post-few-shot prompting (as described in the Methodology section) Gemini performance is presented in Table V. A few-shot prompting provided scaffolding regarding: 1) context (language, terminology), 2) output structure, and 3) points of concern.

However, the method still did not result in automatic score assignment, and further prompting often caused the output to diverge from the context. Also, only in rare cases did the model understand its task on the first try, offering instead the repeating of example instances and similar - even when explicitly instructed to complete the review for the last one. Therefore, we conclude that, when understanding the task, the approach yields significant and automatic output adjustment to established practice, making them more comparable almost effortlessly. Additionally, commentary seems to be more accurate in those cases. Nevertheless, the model still requires scaffolding most of the time, sometimes losing context in the process. To conclude, in the need for quick grading criteria establishment, this approach does work, but the subject models do not appear to keep the context well.

V. CONCLUSION

Research presented in this paper is aimed at assessment of the code review performed by state-of-the-art LLM GAI compared to human graders, and exploration of few-shot prompting in order to boost task performance.

The results suggest that commercial general LLMs can be useful in providing supplementary insight on code, but with required human supervision. Moreover, automatized

TABLE V
PRE- AND POST-FEW-SHOT PROMPTING PERFORMANCE (GEMINI)

	human grader	Gemini zero-shot	Gemini few-shot
commentary	Vaš zadatak je ručno ocijenjen Komentari su sljedeći: - Optimalnije je prvo provjeriti sadrži li broj unesenu znamenku pa onda provjeravati je li prim - nepotrebno više puta provjeravate je li broj sadrži traženu znamenku Za eventualna dodatna pitanja otvorite ticket.	... **Improvements:** **Efficiency:** - Optimize 'jeLiProst' to check divisibility only up to the square root of the number. - Consider using the Sieve of Eratosthenes for larger ranges. **Clarity:** - Use more descriptive variable and function names (e.g., 'isPrime' instead of 'jeLiProst'). - Add comments to explain logic. - Use consistent indentation and spacing. **Modularization:** Break down code into functions for better organization. **Input validation:** Ensure the user enters a valid digit (0-9). **Error handling:** Consider handling invalid input and potential issues. **Output formatting:** Improve readability by adding newlines or formatting prime numbers differently. - Explore alternative prime checking algorithms... - Consider adding more descriptive output ...	Ovo rješenje je ispravno i koristi efikasan algoritam. Prvo se generiraju svi troznamenkasti brojevi. Zatim se provjerava je li broj prim i ako jest, provjerava se sadrži li unesenu znamenku. Ako sadrži, broj se ispisuje i dodaje u zbroj.
score	100	50	n/a

score assignment by those LLMs based on code quality seems prone to error at best and slightly better than random at worst, with multiple possible explanations. Finally, few-shot prompting for the task of code grading better adjusted to preferential criteria can work, albeit in a very limited scope.

The first point, as discussed in the Related Work section, is supported by preceding research. However, this research offers novel insight on automatic score assignment and even goes beyond merely measuring its performance also speculating on possible causes. One of them - low correspondence between user's grading criteria - which is a phenomenon undesirable even concerning solely commentary, served as motivation for an additional experiment, where the few-shot prompting method was tested, in an effort to efficiently scaffold commentary focus. The results suggest that the approach does have a beneficial impact on output features alignment with grading criteria, thus providing a fairer and seemingly more accurate commentary. However, the model easily diverges from the learning context, rendering the method impractical.

ACKNOWLEDGMENT

The authors acknowledge the support of the research project: "Including EVERYone in GREEN Data Analysis" (Strategic Partnership, EVERGREEN, 2022-1-SK01-KA220-HED-000089149) co-funded by the Erasmus+ programme of the European Union.

REFERENCES

- [1] J. Savelka, A. Agarwal, C. Bogart, and M. Sakr, "Large language models (gpt) struggle to answer multiple-choice questions about code," 2023, unpublished.
- [2] J. Leinonen, P. Denny, S. MacNeil, S. Sarsa, S. Bernstein, J. Kim, A. Tran, and A. Hellas, "Comparing code explanations created by students and large language models," 2023.
- [3] A. Tran, L. Li, E. Rama, K. Angelikas, and S. Macneil, "Using large language models to automatically identify programming concepts in code snippets," 06 2023.
- [4] A. H. Mohammadkhani, C. K. Tantithamthavorn, and H. Hemmati, "Explaining transformer-based code models: What do they learn? when they do not work?" 2023 *IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pp. 96–106, 2022.
- [5] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," 2023 *IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1430–1442, 2023.
- [6] N. M. S. Surameery and M. Y. Shakor, "Use chat gpt to solve programming bugs," *International Journal of Information technology and Computer Engineering*, 2023.
- [7] S. Kang, G. An, and S. Yoo, "A preliminary evaluation of llm-based fault localization," 2023, unpublished.
- [8] S. Jiang, Y. Wang, and Y. Wang, "Selfevolve: A code evolution framework via large language models," 2023, unpublished.
- [9] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, "Inferfix: End-to-end program repair with llms," 2023, unpublished.
- [10] N. T. Islam, J. Khoury, A. Seong, G. D. L. T. Parra, E. Bou-Harb, and P. Najafirad, "Llm-powered code vulnerability repair with reinforcement learning and semantic reward," 2024.
- [11] M. Carey, "Code review gpt," <https://github.com/mattzcarey/code-review-gpt/>, accessed: 2023-02-26.
- [12] R. Balse, V. Kumar, P. Prasad, and J. M. Warriem, "Evaluating the quality of llm-generated explanations for logical errors in cs1 student programs," *Proceedings of the 16th Annual ACM India Compute Conference*, 2023.
- [13] A. Hellas, J. Leinonen, S. Sarsa, C. Koutcheme, L. Kujanpää, and J. Sorva, "Exploring the responses of large language models to beginner programmers' help requests," 2023.
- [14] R. Balse, B. Valaboju, S. Singhal, J. M. Warriem, and P. Prasad, "Investigating the potential of gpt-3 in providing feedback for programming assessments," vol. 1, 2023.
- [15] N. Kiesler, D. Lohr, and H. Keuning, "Exploring the potential of large language models to generate formative programming feedback," 2023 *IEEE Frontiers in Education Conference (FIE)*, pp. 1–5, 2023.
- [16] I. Azaiz, O. Deckarm, and S. Strickroth, "Ai-enhanced auto-correction of programming exercises: How effective is gpt-3.5?" *ArXiv*, vol. abs/2311.10737, 2023.
- [17] Anishka, A. Mehta, N. Gupta, A. Balachandran, D. Kumar, and P. Jalote, "Can chatgpt play the role of a teaching assistant in an introductory programming course?" *ArXiv*, vol. abs/2312.07343, 2023.
- [18] C. Pomprasit and C. Tantithamthavorn, "Gpt-3.5 for code review automation: How do few-shot learning, prompt design, and model fine-tuning impact their performance?" 2024, unpublished.
- [19] T. B. Brown *et al.*, "Language models are few-shot learners," vol. 2020-December. Neural information processing systems foundation, 2020.